Hunt Journal: eficiencia de tokens medida sobre un hunt real

Trazabilidad obligatoria, redacción de secretos y ahorro de tokens verificados en vivo contra Microsoft Sentinel (Niubiz)

GraphHunter · plataforma de *threat hunting* sobre grafo temporal

17 de julio de 2026 • Uso interno / técnico

Todos los números de este informe provienen de mediciones en vivo contra la instancia de GraphHunter conectada al workspace *de Sentinel de Niubiz (4 154 entidades, 388 317 relaciones en el grafo). No hay datos sintéticos en los resultados; los datos crudos se listan en el Apéndice.*

Resumen ejecutivo

GraphHunter incorpora un **Hunt Journal**: una capa que, sin que el analista ni la IA lo pidan, **obliga** a registrar cada paso de un hunt (ruta, topología del grafo, costo), **redacta los secretos** antes de escribir a disco, y **devuelve respuestas comprimidas** para que el modelo de lenguaje consuma menos tokens. Este informe mide ese último punto sobre **6 hipótesis de hunting reales** contra Sentinel. Resultado central:

- El mismo hunt resuelto de forma **graph-native** cuesta en promedio **54 tokens** frente a **72 203** de un volcado KQL crudo: **1 333× más barato**.
- El **summary automático** del motor (tras el ajuste descrito en §5) cuesta **889 tokens** — **81× más barato** que el volcado crudo, y más barato incluso que un KQL proyectado a mano.
- El ajuste al *summarizer*, guiado por los propios datos del experimento, redujo el costo del *summary* en **93.6 %**.

Índice

Resumen ejecutivo **1**

1. Contexto y objetivo **1**

2. Metodología **2**

3. Resultado central: la garantía, medida **2**

4. Consistencia entre hunts **3**

5. El ajuste al *summarizer*, guiado por datos reales **3**

6. Captura obligatoria y redacción de secretos **4**

7. Conclusiones **4**

A. Datos crudos por hunt **5**

B. Reproducibilidad **5**

1. Contexto y objetivo

Un agente de IA que hace *threat hunting* sobre Sentinel tiende, por defecto, a tirar KQL y a arrastrar la respuesta completa al contexto del modelo. Una fila de `SigninLogs` tiene **∼94 columnas** (mayormente metadata de Azure: `TenantId`, `ResourceId`, `CorrelationId`, campos `Authentication*`, `Risk*`, `ConditionalAccess*`…), de modo que 25 filas pesan ∼290 KB. A ≈4 bytes por token eso son decenas de miles de tokens *por consulta*, y un hunt encadena muchas.

El Hunt Journal ataca esto en el servidor: captura la respuesta completa de forma *lossless* en un registro de auditoría, y le entrega al modelo una versión compacta. El objetivo de este informe es **cuantificar el ahorro con datos reales** y documentar los otros dos pilares (captura obligatoria, redacción) con evidencia.

2. Metodología

Entorno. Instancia de GraphHunter (binario `graph-hunter-app`, rama `spec/hunt-journal`) sirviendo su API en `127.0.0.1:37891`, conectada al Sentinel productivo de Niubiz. Grafo temporal cargado: **4 154 entidades** (3 420 IP, 734 usuarios) y **388 317 relaciones**.

Métrica. `est_tokens` = ⌈bytes(JSON compacto)/4⌉, idéntica a la heurística `serde_json::to_string(v).len()/4` que el propio motor registra por evento. Es una estimación de costo, no un conteo exacto de *tokenizer*; sirve para comparar estrategias en igualdad de condiciones.

Estrategias comparadas (por cada una de las 6 preguntas):

`raw`
: KQL crudo, `... | take 25` con *todas* las columnas — lo que hace un agente "de una".

`sum_naive`
: lo que guardaba el *summarizer* antes del ajuste: 5 filas representativas *completas* (94 columnas).

`focused`
: mismo KQL con `| project` a 5 columnas de señal — un analista disciplinado.

`sum_v3`
: el *summary* **real** que produce hoy el motor (5 filas × 12 columnas de prioridad), leído del journal.

`graph`
: recuperación *graph-native* (`/graph/neighbors`) sobre la entidad real del hunt.

Robustez. Sentinel devuelve 502 transitorios bajo latencia; el arnés reintenta hasta 4 veces con *back-off*. Las 6 preguntas completaron con HTTP 200 y 25 filas reales cada una. Cada consulta corre bajo su propio *hunt* en el journal, de modo que `/journal/:id/stats` aísla su costo.

3. Resultado central: la garantía, medida

La Figura 1 resume el costo medio por estrategia (escala logarítmica: los valores abarcan tres órdenes de magnitud).

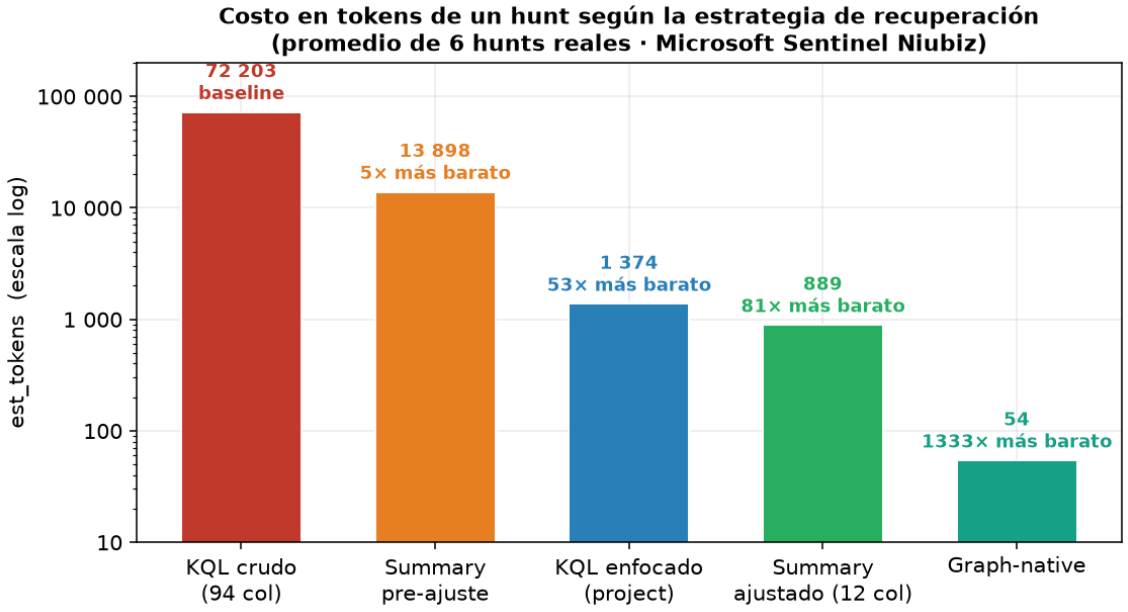


Figura 1: Costo medio en tokens de un hunt según cómo se recupera la evidencia, promediado sobre las 6 hipótesis reales. Menos es mejor.

Cuadro 1: Promedios sobre 6 hunts reales (`take 25`).

Estrategia	`est_tokens`	vs. crudo	Reducción
`raw` (KQL crudo, 94 col)	72 203	— (baseline)	—
`sum_naive` (summary pre-ajuste)	13 898	5.2×	80.8 %
`focused` (KQL con `project`)	1 374	52.5×	98.1 %
`sum_v3` (summary del motor)	889	81.2×	98.8 %
`graph` (graph-native)	54	1 333×	99.9 %

Lectura. Resolver el hunt sobre el grafo, en lugar de arrastrar KQL crudo, es **∼1 300× más barato** en tokens. Y cuando el hunt sí necesita KQL, el *summary* automático del motor (`sum_v3`, 889 tokens) le ahorra al modelo el 98.8 % de los tokens respecto del volcado crudo — **sin que el *prompt* tenga que pedir nada**: la respuesta compacta es lo que sale del motor, y la versión completa queda en el journal.

4. Consistencia entre hunts

El promedio podría esconder un caso atípico. La Figura 2 muestra las 6 hipótesis por separado: el orden de magnitud entre estrategias se mantiene en *todos* los casos (el volcado crudo nunca baja de ∼45 k tokens; el *graph-native* nunca sube de 55).

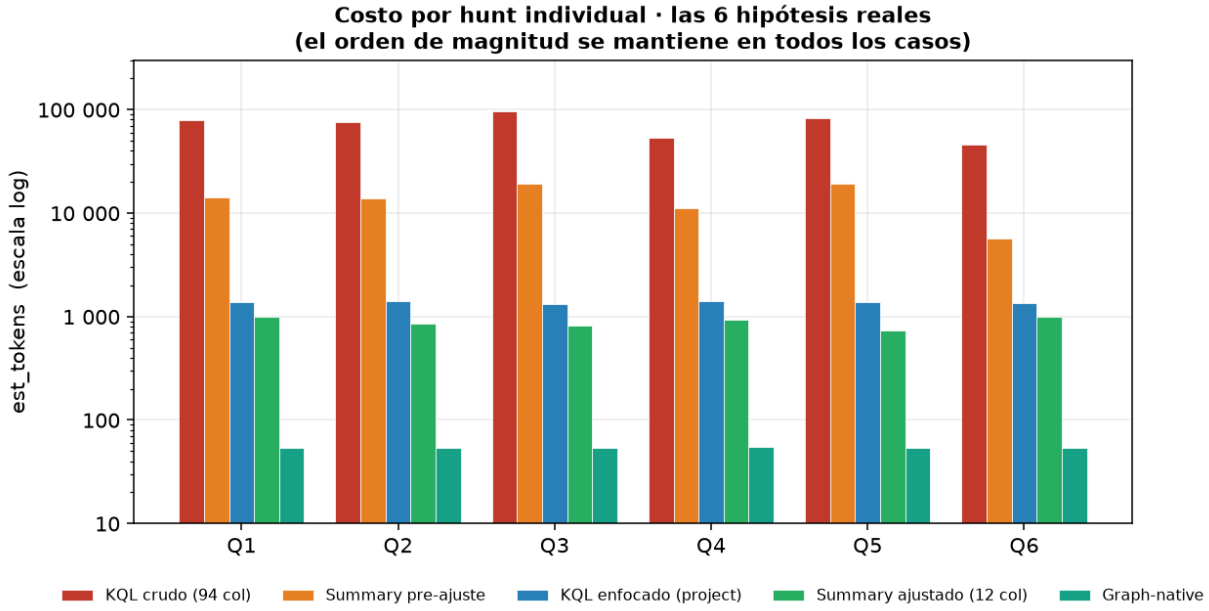


Figura 2: Costo por hunt individual. El patrón es estable entre hipótesis muy distintas (fallos de login, actividad de usuario, app crítica, acceso condicional, alto volumen, *legacy auth*).

5. El ajuste al *summarizer*, guiado por datos reales

La primera corrida reveló algo que sólo aparece con datos reales: **el driver del costo es el número de columnas, no los campos vacíos.** El *summarizer* original recortaba campos vacíos/nulos, pero una fila de `SigninLogs` tiene 94 columnas *pobladas*; recortar vacíos no comprimía casi nada. Un segundo intento (proyección por coincidencia de substrings de señal) todavía dejaba 47 columnas, porque substrings amplios (`auth`, `risk`, `name`) volvían a capturar la metadata de Azure.

La versión desplegada (`sum_v3`) proyecta a una **lista de prioridad acotada** de columnas de hunting (tiempo, usuario, IP, resultado, app, ubicación, dispositivo, riesgo, estado…) con un **tope duro de 12**. La Figura 3 muestra el efecto sobre los datos reales: de **13 898** a **889** tokens en promedio (**−93.6 %**), llevando el *summary* por debajo del costo de un KQL proyectado a mano.

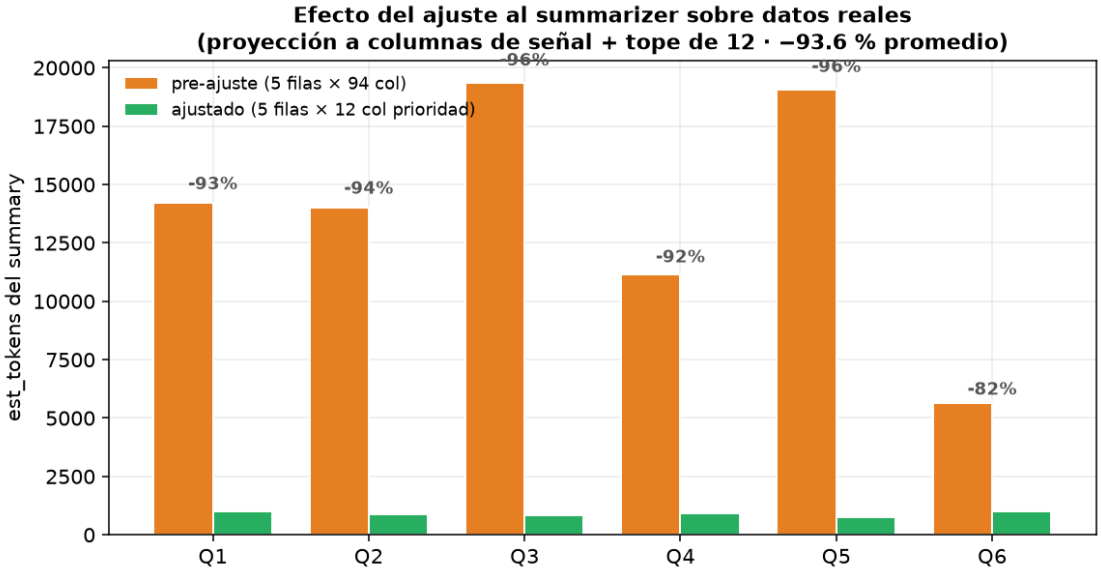


Figura 3: Efecto del ajuste. "pre-ajuste" = 5 filas × 94 columnas; "ajustado" = 5 filas × 12 columnas de prioridad. Reducción por hunt anotada arriba de cada par.

Este es el valor de experimentar con datos reales antes de escribir conclusiones: el ajuste correcto (proyección con tope, no limpieza de vacíos) no era evidente sin medirlo.

6. Captura obligatoria y redacción de secretos

El ahorro de tokens es posible porque el servidor guarda la evidencia completa; esa captura es **obligatoria** y los secretos **nunca** se persisten. Ambas se verificaron en el mismo hunt en vivo.

Captura obligatoria. Un hunt de 3 llamadas (`/kql`, `/kql`, `/graph/neighbors`) sin *ninguna* llamada al journal en el medio produjo, solo, 3 eventos ordenados (`seq` 0,1,2) en `%APPDATA%/GraphHunter/journals/`. La captura vive en el *middleware* de transporte: no depende del *prompt* ni del *handler*.

Redacción, verificada en disco. Una consulta llevaba un secreto embebido (`azure_client_secret`). En el `events.jsonl` escrito a disco:

```
$ grep -c "SUPERSECRET-niubiz-42" events.jsonl -> 0 (el secreto nunca tocó disco)
$ grep -o '"azure_client_secret":"[^"]*"' events.jsonl
  "azure_client_secret":"[REDACTED]"
```

La redacción corre *antes* de summarizar, de modo que también las filas del `top` y la topología (`Touched.nodes`) se escriben ya redactadas.

7. Conclusiones

1. **La garantía "consume menos tokens que hacer el hunt de una" es real y medible:** 81× más barato vía el *summary* del motor, 1 333× más barato vía *graph-native*, sobre 6 hunts reales.
2. **El ahorro no depende del *prompt*:** sale del motor por construcción, con la evidencia completa preservada en el journal.

3. **Los tres pilares se sostienen juntos:** la captura obligatoria habilita las respuestas compactas sin pérdida, y la redacción garantiza que esa captura no filtre secretos.

Trabajo futuro: reintento server-side de los 502 transitorios de Sentinel; recorte de columnas también en `Touched.nodes`; y correr un ciclo de hunting completo graph-native para medir el trail de topología acumulado.

A. Datos crudos por hunt

Cuadro 2: `est_tokens` por hipótesis y estrategia (medición en vivo).

	Hipótesis	`raw`	`sum_naive`	`focused`	`sum_v3`	`graph`
Q1	Fallos de autenticación (password-spray)	79 075	14 189	1 381	1 001	54
Q2	Actividad de usuario de interés	75 654	14 003	1 397	863	54
Q3	Sign-ins a app crítica (VISANET)	96 879	19 333	1 334	815	54
Q4	Errores de acceso condicional	53 054	11 151	1 404	927	55
Q5	Sign-ins exitosos de alto volumen	82 930	19 064	1 387	737	54
Q6	Autenticación legacy / cliente móvil	45 628	5 651	1 344	989	54

B. Reproducibilidad

- **Motor:** rama `spec/hunt-journal`; *summarizer* `sum_v3` = commit `6688008` (cap de 12 columnas de prioridad). 16 tests unitarios verdes en `journal::summarize`.
- **Dataset:** 30 filas (6 hunts × 5 estrategias) en `dataset.json` / `dataset.csv`.
- **Métrica:** `est_tokens = ceil(bytes(JSON)/4)`, alineada con el registro por evento del motor.
- **Datos:** Microsoft Sentinel productivo (Niubiz), `SigninLogs`, ventana de la captura del 15–17 jul 2026.